EXPERIMENT 4: CLOCK SYNCHRONIZATION ALGORITHMS

Date Of Performance:

Date of Submission:

1. AIM

To study clock synchronization algorithms in distributed systems and understand how they maintain a consistent and accurate notion of time among multiple nodes.

1.1 OBJECTIVES

1. To understand the importance of clock synchronization in distributed systems.
2. To study the causes of clock drift and clock skew.
3. To analyze different clock synchronization algorithms and their working principles.
4. To compare internal and external clock synchronization methods.
5. To understand logical and physical clock concepts.
6. To evaluate the advantages and limitations of various synchronization algorithms.
7. To study the role of synchronization in event ordering and consistency.

1.2 SCOPE

1. Understanding Clock Synchronization and Time Differences

The scope of clock synchronization algorithms includes understanding the importance of maintaining a consistent notion of time across distributed systems. Since multiple computers operate independently, their system clocks may not remain perfectly synchronized. Factors such as clock drift and clock skew cause time differences between machines. Studying these concepts helps in identifying the reasons for time inconsistencies and understanding why synchronization mechanisms are necessary for proper coordination among distributed nodes.

2. Study of Clock Synchronization Algorithms

Another important scope is analyzing different clock synchronization algorithms and understanding their working principles. Various algorithms such as Cristian's Algorithm, Berkeley Algorithm, and Network Time Protocol (NTP) are designed to synchronize clocks across systems. This study also includes comparing internal synchronization methods, which focus on maintaining consistency among system clocks, and external synchronization methods, which align system clocks with an external time source such as UTC. Understanding these techniques helps in selecting appropriate synchronization methods for distributed environments.

3. Logical and Physical Clock Concepts in Distributed Systems

The scope also involves understanding the concepts of logical clocks and physical clocks used in distributed computing. Physical clocks represent actual system time, while logical clocks are used to maintain the order of events in distributed systems. Evaluating the advantages and limitations of different synchronization algorithms helps in identifying suitable techniques for maintaining accurate time and coordination among distributed processes.

4. Role of Synchronization in Event Ordering and System Consistency

Clock synchronization plays a critical role in maintaining correct event ordering and consistency in distributed applications. By synchronizing system clocks, distributed processes can coordinate actions and maintain proper sequencing of events. This is particularly important in applications such as distributed databases, cloud computing systems, transaction processing, and real-time systems where accurate timing and coordination are essential for reliable operation.

2 LITERATURE SURVERY

Clock synchronization is a fundamental problem in distributed systems because independent clocks on different nodes drift due to hardware imperfections, environmental influences, and communication delays. Without synchronization, actions on distributed nodes may have inconsistent timestamps, leading to problems in event ordering, coordination, and system consistency. Several research works have addressed this problem using both physical and logical synchronization techniques.

A very recent survey by *Dissanayake and Algama (2024)* provides a detailed analysis of core clock synchronization algorithms with emphasis on message complexity and performance. This paper discusses classic algorithms like Lamport's logical clocks, Vector Clocks, Cristian's algorithm, and Berkeley algorithm, comparing how many messages each algorithm uses to achieve synchronization—a key performance metric in distributed environments. It highlights that while logical clock methods help determine event order, synchronization protocols like Cristian's and Berkeley's focus on minimizing clock offsets and drift across networked nodes.

In addition to the above, broader surveys on time synchronization such as the "Evolution of network time synchronization towards nanoseconds accuracy" (2022) examine how synchronization approaches have advanced over time, especially for high-precision requirements. This work reviews a range of techniques from standard protocols like Network Time Protocol (NTP) to more recent high-accuracy approaches, showing how research has evolved with network complexity and performance needs.

Secure time synchronization is another emerging research area especially in sensor networks and IoT. The survey "A Survey of Secure Time Synchronization" (2023) discusses not only basic synchronization protocols and challenges but also security vulnerabilities that attack time synchronization itself (e.g., delay and replay attacks) and research directions to secure these protocols, which is crucial in distributed and cyber-physical systems.

Finally, specialized algorithmic research such as the HyNTP hybrid synchronization algorithm (2021) and the skewless synchronization model (2014) focus on improving convergence speed and robustness against drift and timing loops. These works propose refined mathematical models and synchronization logic to improve clock alignment even under adverse network conditions and dynamic topologies.

2.1 THEORY:

Clock Synchronization:
Clock synchronization is a fundamental concept in distributed systems that ensures all participating computers or processes maintain a consistent notion of time. In a distributed environment, each system has its own local clock, which may differ from others due to hardware imperfections, clock drift, and varying execution speeds. Since there is no global clock shared among all systems, synchronization mechanisms are required to coordinate time-dependent operations.

Clock synchronization is essential for maintaining correct event ordering, consistency in distributed databases, coordination of tasks, fault detection, and logging. Without synchronization, it becomes difficult to determine the correct sequence of events, which may lead to data inconsistency and system errors. Distributed systems therefore rely on clock synchronization algorithms to minimize time differences among nodes.

Concept of Clock Synchronization in Distributed Systems:
In a distributed system, multiple nodes or processes execute concurrently and communicate through a network. Each node maintains its own clock, and due to differences in clock rates, the clocks may gradually drift apart. Because of this drift, timestamps generated by different nodes may not accurately reflect the real order of events.

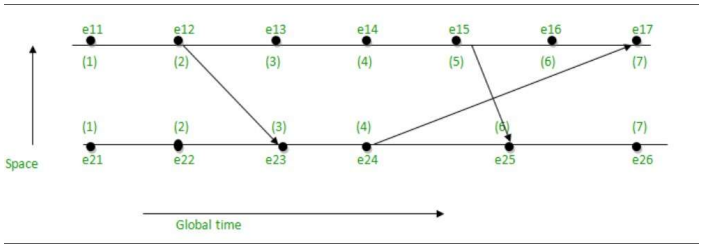
To overcome this problem, clock synchronization algorithms are used. These algorithms allow nodes to exchange timing information and adjust their clocks accordingly. Synchronization may be achieved using a central time server or through cooperation among all nodes in the system.

In a typical synchronization scenario, a process may synchronize its clock with one or more reference clocks using message exchanges. These messages help estimate network delays and clock offsets, allowing the system to compute an adjusted time that is closer to a common reference.

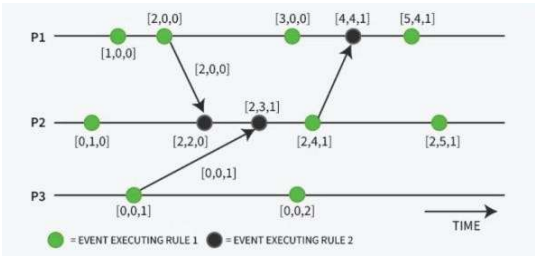
Logical Clocks

Logical clocks are used to maintain the ordering of events in a distributed system without relying on real-time clocks. They do not represent actual physical time but provide a logical sequence in which events occur.

Lamport's logical clock assigns a numerical timestamp to each event. The timestamp increases with every event and is updated when messages are sent or received. This helps establish a "happened-before" relationship between events.



Vector Logical clocks extend this concept by maintaining a vector of timestamps, one for each process. Vector clocks provide more accurate information about causality between events and help detect concurrent events in distributed systems.



Physical Clock Synchronization

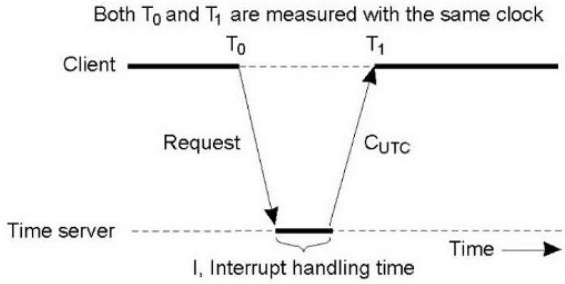
Physical clock synchronization focuses on synchronizing real-time clocks across different systems. These clocks are influenced by clock drift and network delays, so synchronization algorithms aim to reduce time differences within an acceptable bound.

Physical clock synchronization can be classified into internal and external synchronization. Internal synchronization ensures that clocks within a system are consistent with each other, while external synchronization aligns system clocks with an external reference such as UTC.

Cristian's Algorithm

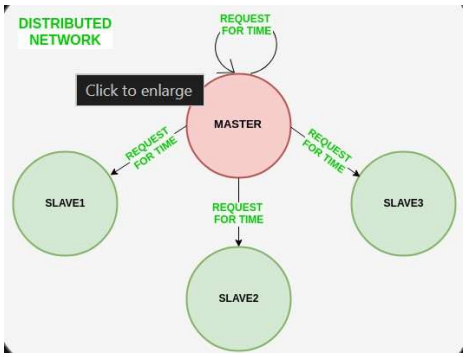
Cristian's algorithm is a client-server based clock synchronization technique. In this method, a client requests the current time from a time server. The server responds with its timestamp, and the client estimates the network delay to adjust its clock.

This algorithm is simple and effective in environments with low and predictable network latency. However, its accuracy depends heavily on the delay symmetry and reliability of the time server.



Berkeley Algorithm

The Berkeley algorithm is used when no accurate external time source is available. In this approach, one node acts as a master and periodically polls other nodes to obtain their clock values.The master calculates an average time after excluding faulty or outlier clocks and then sends clock adjustment values to each node. This ensures internal synchronization among all participating nodes. The Berkeley algorithm is suitable for local-area networks but may not scale well in large or highly dynamic systems.



Why Clock Synchronization is Required?

Clock synchronization is required for several important reasons:

- **Event Ordering:** Ensures correct sequence of events across distributed processes.
- **Data Consistency:** Maintains consistency in distributed databases and replicated systems.
- **Coordination:** Enables synchronized execution of tasks in distributed applications.
- **Fault Detection:** Helps detect failures by comparing timestamps.
- **Security:** Supports time-based authentication and logging mechanisms.
- **Real-Time Applications:** Essential for systems with strict timing constraints.

2.2 ALGORITHM:

Algorithm 1: Lamport's Logical Clock Algorithm (Logical Clock)

INPUT : Events occurring in distributed processes
OUTPUT : Logical timestamp for each event
BEGIN:

1. Initialize logical clock LC = 0 for each process
2. For every event e in process Pi do
   LC ← LC + 1
   Timestamp(e) ← LC
   End For
3. When process Pi sends message m
   LC ← LC + 1
   Attach timestamp T = LC with message m
   Send m
4. When process Pj receives message m with timestamp T
   LCj ← max(LCj, T) + 1
   Timestamp(receive event) ← LCj
5. Continue execution for next events
   END

Algorithm 2: Vector Clock Algorithm (Logical Clock)
INPUT : Events and messages among processes
OUTPUT : Vector timestamp representing causal relationship

BEGIN
1. For each process Pi
   Initialize vector clock VC[i] = 0 for all i
2. Before executing any event in process Pi
   VC[i] ← VC[i] + 1
3. When process Pi sends message m
   Attach vector clock VC with message m
   Send m
4. When process Pj receives message m with vector clock V
   For each k from 1 to n
   VCj[k] ← max(VCj[k], V[k])
   End For
   VCj[j] ← VCj[j] + 1
5. Assign updated vector clock to the receive event
END

Algorithm 3: Cristian's Clock Synchronization Algorithm (Physical Clock)
INPUT : Server time TS
OUTPUT : Adjusted client clock time
BEGIN
1. Client records local time T0
2. Client sends request to time server
3. Server records its current time TS
   Server sends TS to client
4. Client receives reply at time T1
5. Compute round trip delay
   D ← (T1 - T0)
6. Estimate one way delay
   Delay ← D / 2
7. Adjust client clock
   Client_Time ← TS + Delay
8. Update local clock
END

Algorithm 4: Berkeley Clock Synchronization Algorithm (Physical Clock)
INPUT : Clock values of all processes
OUTPUT : Synchronized clock time for all systems
BEGIN
1. Select one process as Master
2. Master requests current clock time from all slave processes
3. Each slave sends its local clock value Ti
4. Master collects all clock values
5. Compute average time
   Avg ← (Tmaster + Σ Ti) / N
6. For each process Pi
   Adjustment[i] ← Avg - Ti
7. Master sends adjustment value to each process
8. Each process updates its clock
   Ti ← Ti + Adjustment[i]
END

2.3 PROGRAMMING LANGUAGE:

C Programming Language

The experiment is implemented using the C programming language to simulate clock synchronization and logical time management algorithms used in distributed systems.
The program demonstrates Lamport Logical Clock, Vector Clock, Cristian's Algorithm, and Berkeley Algorithm using simple variables and arrays to represent process clocks and message passing.

Although real distributed systems require network communication and operating system support, this implementation provides a conceptual and educational simulation of clock synchronization techniques in a single execution environment using standard C constructs.

2.3.1 Libraries Used

1) stdio.h

Provides standard input and output functions such as printf(), scanf(), and fgets() for displaying messages and reading user input.

2) stdlib.h

Provides general utility functions such as exit() – used for clean program termination if required. Also supports memory-related and standard utility operations.

2.3.2 Constants Used

1) MAX_PROCESSES

#define MAX_PROCESSES 3

Defines the maximum number of processes used in vector clock and Berkeley algorithm simulations.
Helps in maintaining uniform array sizes and improving code readability.

2.3.3 Functions / Methods Used

1) lamportClock()

Simulates the Lamport Logical Clock algorithm.

- Each event increments the local logical clock.
- When a message is received, the receiver updates its clock using:
- LC = max(local_clock, received_time) + 1
- Demonstrates causal ordering of events in distributed systems.

2) vectorClock()

Implements the Vector Clock algorithm.

- Uses integer arrays to represent vector timestamps.
- On message receipt, the receiving process updates each entry by taking the maximum value.
- Allows detection of causality and concurrency between events.

3) cristianAlgorithm()

Simulates Cristian's Clock Synchronization Algorithm.

- A client requests the current time from a server.
- Network delay is estimated using:
- delay = (T2 - T0) / 2
- The client adjusts its clock using the server's timestamp and estimated delay.

4) berkeleyAlgorithm()

Implements the Berkeley Clock Synchronization Algorithm.

- A master process collects clock values from all participating processes.
- Computes the average time.
- Each process adjusts its clock based on the difference between its time and the average.

5) main()

Acts as the driver function.

- Displays the experiment title.
- Calls all clock synchronization algorithm functions sequentially.
- Marks the completion of the simulation.

2.3.4 Variables Used

- Logical Clock Variables:
  LC1, LC2 – represent Lamport logical clocks of processes.
- Vector Clock Arrays:
  VC1[], VC2[] – represent vector timestamps of processes.
- Time Variables:
  T0, T1, T2 – represent request, server, and response times in Cristian's algorithm.
- Clock Array:
  clocks[] – stores individual process clocks for Berkeley algorithm.

3. IMPLEMENTATION:

Programming Language Used: **C programming.**

Server Used: https://www.programiz.com/c-programming/online-compiler/

3.1 Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_PROCESSES 3

/* ---------------- Lamport Logical Clock ----------------- */
void lamportClock() {
   int LC1 = 0, LC2 = 0;
   int msgTime;
   printf("\n--- Lamport Logical Clock --- \n");
   LC1++; // Event at Process 1
   printf("Process 1 Event, Clock = %d\n", LC1);
   msgTime = LC1; // Send message
   LC2 = (LC2 > msgTime ? LC2 : msgTime) + 1;
   printf("Process 2 receives message, Clock = %d\n", LC2);
}

/* ----------------- Vector Clock ----------------- */
void vectorClock() {
   int VC1[MAX_PROCESSES] = {0, 0, 0};
   int VC2[MAX_PROCESSES] = {0, 0, 0};
   printf("\n--- Vector Clock ---\n");
   VC1[0]++; // Event at Process 1
   printf("Process 1 Vector Clock: [%d %d %d]\n", VC1[0], VC1[1], VC1[2]);
   // Send vector clock to Process 2
   for (int i = 0; i < MAX_PROCESSES; i++) {
      VC2[i] = (VC2[i] > VC1[i]) ? VC2[i] : VC1[i];
   }
   VC2[1]++;
   printf("Process 2 Vector Clock: [%d %d %d]\n", VC2[0], VC2[1], VC2[2]);
}

/* ---------------- Cristian's Algorithm ----------------*/
void cristianAlgorithm() {
   int T0 = 100; // Client request time
   int T1 = 110; // Server reply time
   int T2 = 120; // Client receive time
   int delay = (T2 - T0) / 2;
   int adjustedTime = T1 + delay;
   printf("\n--- Cristian's Algorithm---\n");
```

```
    printf("Client sends request at T0 = %d\n", T0);
    printf("Server replies with time T1 = %d\n", T1);
    printf("Client receives reply at T2 = %d\n", T2);
    printf("Adjusted Client Time = %d\n", adjustedTime);
}

/* ------------------ Berkeley Algorithm ------------------ */
void berkeleyAlgorithm() {
    int clocks[MAX_PROCESSES] = {100, 105, 95};
    int sum = 0, avg;
    printf("\n--- Berkeley Algorithm ---\n");
    printf("Clock values:\n");
    for (int i = 0; i < MAX_PROCESSES; i++) {
        printf("Process %d Clock = %d\n", i + 1, clocks[i]);
        sum += clocks[i];
    }
    avg = sum / MAX_PROCESSES;
    printf("Average Time = %d\n", avg);
    for (int i = 0; i < MAX_PROCESSES; i++) {
        printf("Process %d adjusts clock by %d\n",
               i + 1, avg - clocks[i]);
    }
}
/* ------------------ Main Function ------------------ */
int main() {
    printf("CLOCK SYNCHRONIZATION ALGORITHMS SIMULATION\n");
    lamportClock();
    vectorClock();
    cristianAlgorithm();
    berkeleyAlgorithm();
    printf("\nSimulation Completed.\n");
    return 0;
}
```

OUTPUT:

```
--- Lamport Logical Clock ---
Process 1 Event, Clock = 1
Process 2 receives message, Clock = 2

--- Vector Clock ---
Process 1 Vector Clock: [1 0 0]
Process 2 Vector Clock: [1 1 0]

--- Cristian's Algorithm ---
Client sends request at T0 = 100
Server replies with time T1 = 110
Client receives reply at T2 = 120
Adjusted Client Time = 120
```

```
--- Berkeley Algorithm ---
Clock values:
Process 1 Clock = 100
Process 2 Clock = 105
Process 3 Clock = 95
Average Time = 100
Process 1 adjusts clock by 0
Process 2 adjusts clock by -5
Process 3 adjusts clock by 5

Simulation Completed.
```

CONCLUSION:

Clock synchronization is a fundamental requirement in distributed systems to ensure consistent timing and correct event ordering across multiple processes. Logical clocks like Lamport and Vector clocks help maintain causal relationships, while physical clock algorithms like Cristian's and Berkeley ensure actual time alignment among nodes. These algorithms prevent data inconsistency, enable coordination, and support efficient resource sharing. Simulations of these algorithms help understand their working principles and limitations. Proper synchronization is crucial for applications such as distributed databases, real-time systems, and networked applications. Overall, studying these algorithms provides a strong foundation for designing reliable and synchronized distributed systems.

REFERENCES:

1. Leslie Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, 1978.
2. Andrew S. Tanenbaum and Maarten van Steen, *Distributed Systems: Principles and Paradigms*, Pearson Education.
3. George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair, *Distributed Systems: Concepts and Design*, Pearson Education.
4. Cristian, F., "Probabilistic Clock Synchronization," *Distributed Computing*, Springer, 1989.

SIGN AND REMARKS:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature